

Architecture & Ecosystem Design —

Part 3: Product Boundaries & Ecosystem Structure

SUMMARY

The AROH ecosystem is composed of one Platform and multiple independent Products. The Platform exists to provide reusable capabilities. Products exist to solve domain-specific problems. The ecosystem should feel like one operating environment rather than many disconnected applications.

ECOSYSTEM STRUCTURE PHILOSOPHY

The AROH ecosystem is composed of one Platform and multiple independent Products. The Platform exists to provide reusable capabilities. Products exist to solve domain-specific problems. The Platform should continue to grow in capability. Products should continue to grow in functionality. These are different responsibilities. The ecosystem should never evolve into multiple applications each implementing their own authentication, user management, wallet, notifications, analytics, AI integration, themes, UI libraries, or infrastructure. Every product should appear unique in purpose but identical in platform experience. The ecosystem should feel like one operating environment rather than many disconnected applications.

PLATFORM VS PRODUCT

Everything inside AROH belongs to one of two categories: Platform or Product. There should never be a third category.

Platform owns: Authentication, Authorization, Identity, Profiles, Wallet, Membership, Notifications, Analytics, Configuration, AI, Registry, Search, Documentation, SDK, Design System, Motion System, Themes, Shared APIs, Monitoring, Logging, Audit, Infrastructure Contracts, Feature Flags, Developer Experience.

Products own: Business Logic, Workflows, Domain Models, Specialized UI, Domain Services, Product Settings, Domain Reports, Product-specific AI, Specialized Search Filters, Product Assets, Product Documentation, Product Navigation, Product Permissions (through Platform).

Nothing should exist without clear ownership.

INDEPENDENCE

Every product must be independently understandable. Removing Nebula should never affect JavaPath. Removing JavaPath should never affect SpeDex. Removing Music Mirror should never affect Task Pro. The Platform continues operating. The ecosystem continues functioning. Products may depend on the Platform. Products never depend on each other.

PRODUCT ISOLATION

Every product should be isolated in terms of: Business Logic, Routing, Pages, Components, Assets, Documentation, Testing, Configuration, Domain Events, Release Cycle, Versioning, Internal Architecture. Products should never import code from another product. Instead: Product ↓ Platform ↓ Shared Service ↓ Another Product (Indirectly).

PLATFORM CONTRACTS

Every interaction happens through contracts. Contracts define: Inputs, Outputs, Validation, Errors, Permissions, Events, Responses, Version. Contracts hide implementation. Products should never know: Database Schema, Infrastructure, Provider Logic, Storage Method, Internal Algorithms. This allows platform evolution without product rewrites.

PRODUCT COMMUNICATION

Products should communicate only through the Platform. Never: Nebula ↓ JavaPath. Instead: Nebula ↓ Platform ↓ JavaPath.

Examples: Nebula sends notification ↓ Notification Service ↓ User. Task Pro requests identity ↓ Identity Service ↓ Response. Music Mirror charges wallet ↓ Wallet Service ↓ Transaction. SpeDex requests AI ↓ AI Platform ↓ Provider. This architecture eliminates tight coupling.

SHARED USER JOURNEY

One Account. One Login. One Profile. One Wallet. One Membership. One Theme. One Notification Center. One Search. One AI. One Configuration. Many Products. The user never creates multiple accounts, never manages multiple wallets, never changes settings repeatedly. Everything follows the user.

CROSS-PRODUCT EXPERIENCE

Cross-product transitions should feel seamless. Authentication persists. Theme persists. Navigation remains familiar. Profile remains identical. Notifications remain centralized. Wallet updates immediately. Membership updates instantly. Analytics continue. AI remembers context (future). No unnecessary login screens. No duplicate onboarding. No repeated profile setup.

PRODUCT LIFECYCLE

Every product follows a standard lifecycle: Idea ↓ Architecture Review ↓ Platform Dependency Review ↓ Repository Creation ↓ Shared Integration ↓ Development ↓ Testing ↓ Documentation ↓ Release ↓ Monitoring ↓ Maintenance ↓ Expansion ↓ Retirement (if required). Platform services remain stable throughout.

PRODUCT ONBOARDING

Every new product should require only: Repository Folder, Configuration, Metadata, Routes, Domain Logic, Product Assets, Documentation, Platform Registration. No authentication implementation. No wallet implementation. No analytics implementation. No notifications. No design system creation. No SDK recreation. Platform already provides these.

PRODUCT REGISTRATION

Every product should register itself. Registry includes: Product ID, Display Name, Description, Version, Status, Category, Icon, Logo, Banner, Owner, Dependencies, Supported Platforms, Permissions, Membership Requirements, Navigation, Documentation, Search Metadata, Analytics ID, AI Support, Release Channel, Health Status, Future Marketplace Metadata. Homepage product cards should be generated from this registry, never hardcoded.

PRODUCT CATEGORIES

Products should belong to categories: Education, Productivity, Developer Tools, AI, Media, Finance, Utilities, Collaboration, Analytics, Platform, Experimental. Future categories can be added. Categories improve: Discovery, Filtering, Search, Recommendations, Navigation, Analytics.

DOMAIN BOUNDARIES

Every product owns one primary domain: Nebula — Knowledge; JavaPath Pro — Programming Education; SpeDex — Speed-based Utility Domain; Music Mirror — Music; Task Pro — Task Management; Platform — Infrastructure. Domain overlap should be minimized. If overlap becomes reusable, move it into the Platform.

DEPENDENCY RULES

Allowed: Product ↓ Platform; Platform ↓ Infrastructure; SDK ↓ Platform.

Not Allowed: Product ↓ Product; Infrastructure ↓ Product; UI ↓ Database; Database ↓ UI; Circular Dependencies; Shared Mutable State Between Products.

NAVIGATION ARCHITECTURE

Platform Navigation: Home, Explore, Products, AI, Documentation, Updates, Account, Settings, Help. Product Navigation: Unique to each product. Platform navigation remains familiar. Product navigation changes.

SHARED DESIGN LANGUAGE

Regardless of product: Typography identical. Spacing identical. Motion identical. Buttons identical. Forms identical. Cards identical. Dialogs identical. Colors identical. Accessibility identical. Users should instantly recognize they are inside AROH.

SHARED CONFIGURATION

Products inherit: Theme, Localization, Date Format, Currency, Accessibility, Notification Preferences, Privacy Settings, Feature Flags, AI Preferences, Performance Settings. Only product-specific settings remain local.

SHARED ANALYTICS

Every product contributes analytics. Platform combines them. Allows: User Journey, Retention, Feature Adoption, Cross-product Usage, Popular Products, Growth Trends, AI Usage, Wallet Activity, Membership Growth, Platform Health. Analytics should measure the ecosystem, not isolated products.

SHARED AI CONTEXT

Future AI should understand: User, Membership, Products Used, History, Preferences, Goals, Recent Activity, Permissions, Context. Without requiring each product to rebuild memory. The Platform owns context. Products contribute context.

SHARED SEARCH

One search. Multiple products. Documentation. Settings. Commands. Help. Products. AI. Recent Files (future). Recent Activity (future). Everything searchable from one place.

PRODUCT DEPLOYMENT

Products should remain independently deployable. Platform services remain shared. Deployment failure in one product should never break another. Version upgrades should be independent. Rollback should be isolated. Monitoring should identify product-specific failures.

FUTURE ECOSYSTEM EXPANSION

Future additions include: Nebula, JavaPath Pro, SpeDex, Music Mirror, Task Pro, Developer Portal, Marketplace, Plugin Ecosystem, Extension Framework, Enterprise Suite, Admin Suite, Mobile Apps, Desktop Apps, Partner

Products, Third-party Integrations, Cloud Services. Every future addition should integrate through existing platform contracts instead of introducing parallel infrastructure.

ECOSYSTEM EVOLUTION STRATEGY

The ecosystem should evolve by strengthening the Platform first. Every new reusable capability increases engineering leverage. Every new product should require less foundational work than the previous one. The Platform continuously compounds in value while products remain focused on innovation within their own domains. Success is achieved when adding a completely new product requires minimal infrastructure work because authentication, identity, wallet, analytics, AI, notifications, search, design system, SDK, configuration, monitoring, documentation, and operational tooling already exist as mature platform services. At that stage, AROH ceases to be perceived as a collection of applications and becomes a true software ecosystem where the Platform is the permanent foundation and every product is simply another extension of its capabilities.